Task A2 – Don't Read What You Don't Need (5 Marks, K4)

> Part A asks for design, pseudocode and reasoning, so the answer is in pseudocode and tables. The Java in the *Alternative* sections is only for illustration. Items marked `<fill>` need your own observations.

A2.1 – Predict the cost

A 50 MB file contains 50 × 1,024 × 1,024 = **52,428,800 bytes**. `FileInputStream.read()` returns one byte per call, so the program makes **about 52.4 million `read()` calls** (52,428,801 including the final –1). `FileInputStream` is unbuffered: each `read()` goes through the JNI layer and asks the operating system for one byte (a system call), and every system call costs a switch between user mode and kernel mode (typically hundreds of nanoseconds to microseconds). The work is dominated by this call overhead, not by disk speed, because the file is read sequentially and the OS has the data cached. Prediction: if each call costs ~0.3 µs, 52.4 million calls take about 15 s, far slower than the disk could deliver (50 MB takes well under a second at normal disk speeds). Time grows linearly with file size, O(n) calls.

A2.2 – Redesign using buffering

```
// Read in blocks of 8 KB and scan the block in memory
byte[] buf = new byte[8192];
long lines = 0;
try (FileInputStream in = new FileInputStream(file)) {
    int n;
    while ((n = in.read(buf)) != -1)
        for (int i = 0; i < n; i++) if (buf[i] == '\n') lines++;
}
// or:  new BufferedInputStream(new FileInputStream(file))   (default
8 KB buffer) and keep calling read()
```

```
import java.io.*;

public class LineCounter {
    /** Unbuffered: one read() call (one system call) per byte. */
    public static long countUnbuffered(String file) throws IOException
{
        long lines = 0;
        try (FileInputStream in = new FileInputStream(file)) {
            int b;
            while ((b = in.read()) != -1) if (b == '\n') lines++;
        }
        return lines;
    }

    /** Buffered: reads 8 KB blocks, then scans them in memory. */
    public static long countBuffered(String file) throws IOException {
        long lines = 0;
        byte[] buf = new byte[8192];
```

```
        try (FileInputStream in = new FileInputStream(file)) {
            int n;
            while ((n = in.read(buf)) != -1)
                for (int i = 0; i < n; i++) if (buf[i] == '\n') lines++;
        }
        return lines;
    }

    /** Same idea using BufferedInputStream. */
    public static long countBufferedStream(String file) throws IOException {
        long lines = 0;
        try (BufferedInputStream in = new BufferedInputStream(new FileInputStream(file))) {
            int b;
            while ((b = in.read()) != -1) if (b == '\n') lines++;
        }
        return lines;
    }
}
```

Effect on operating-system calls: with an 8,192-byte buffer, each OS read delivers up to 8,192 bytes, so the number of system calls falls from 52,428,800 to 52,428,800 / 8,192 = **6,400** (a reduction by a factor of 8,192). The per-byte work is now a simple array access in memory. `BufferedInputStream` also reduces system calls to 6,400, but still makes one (synchronised) method call per byte, so it is faster than unbuffered but slower than scanning a `byte[]` yourself. These timings are measured in Task B1.3.

A2.3 – When buffering does not help

- When the program already reads in large blocks, e.g. `in.read(new byte[1_000_000])` or `Files.copy`: the buffer adds an extra copy and no benefit.
- When access is random (frequent `seek`s): each seek throws away the buffer, so every read refills it.
- When the data is already in memory (`ByteArrayInputStream`) or the source is a device where each message must be delivered immediately (interactive socket traffic that is flushed after every message).

Alternative concept and complexity

`Files.readAllLines(path)` reads the whole file into a `List<String>`:

```
List<String> lines = Files.readAllLines(Path.of(file));
long count = lines.size();
```

- **Time:** O(file size) for both approaches; every byte must be read once. `readAllLines` also decodes characters and creates one `String` object per line, so it has a higher constant factor than counting newlines in a byte buffer.

- **Memory:** the buffered stream keeps only the buffer: **O(buffer size)**, e.g. 8 KB, independent of file size. `readAllLines` keeps the whole file as Strings: **O(file size)**. For 50 MB of ~36-character lines (about 1.46 million lines) that is far more than 50 MB of heap, because every `String` object has header and array overhead; with a small heap it can end in `OutOfMemoryError`.
- **When to use which:** `readAllLines` (or `Files.lines`) is appropriate for small files (a few MB) that must be processed several times or need random access to lines. A buffered stream (or `Files.lines` as a lazy stream) is appropriate for large files and one-pass work such as counting.